Journal of Physics:
Conference Series

PURPOSE-LED PUBLISHING™

PAPER • OPEN ACCESS

Design and Implementation of a Direct Memory Access ControllerBased on Microcontroller Unit

To cite this article: Chao Lu *et al* 2022 *J. Phys.: Conf. Ser.* **2221** 012016

View the article online for updates and enhancements.

You may also like

- Algorithm and hardware considerations for real-time neural signal on-implant processing
Zheng Zhang, Oscar W Savolainen and Timothy G Constandinou

- Impact of incident direction on neutron-induced single-bit and multiple-cell upsets in 14 nm FinFET and 65 nm planar SRAMs
Shao-Hua Yang, , Zhan-Gang Zhang et al.

- Influences of supply voltage on single event upsets and multiple-cell upsets in nanometer SRAM across a wide linear energy transfer range
Yin-Yong Luo, , Wei Chen et al.

This content was downloaded from IP address 14.241.191.19 on 17/02/2026 at 16:03

Design and Implementation of a Direct Memory Access ControllerBased on Microcontroller Unit

Chao Lu*, Haosen Yang, Qi Wu

School of Microelectronics, Xidian University, Xi' an, Shaanxi, China

email: chlu@stu.xidian.edu.cn

Abstract: With increasing IoT technology, the application of MCU is becoming more and more extensive, and higher requirements are put forward for the functions and data processing capabilities of MCU. This trend means the amount of data that the MCU needs to process in the system has increased sharply, so this paper proposes a hardware design scheme for a DMA controller based on the AHB-Lite protocol to replace the MCU to transfer data to improve system performance. The controller supports 8 transfer channels and adopts the method of fixed priority level arbitrating to select channels. The channel source and destination data width and burst type are configurable. In the process of data burst transfer, the condition of crossing the 1K address boundary is taken into consideration. At the same time, the design adopts the method that 8 transfer channels share a buffer FIFO to optimize the area of the controller.

1. Introduction

As the diversification of application scenarios, and the functions of MCUs have become more complex, the amount of data that to be processed in the chip is increasing sharply. The efficiency of data transfer among various modules has become a bottleneck in improving the performance of the MCU. In order to improve system performance, it is necessary for a DMA (Direct Memory Access) controller to replace the CPU of completion a large amount of data transfer operations [1].

DMA is a kind of technology that data can transfer among memory, input and output devices [2]. DMA plays an important role in the system, for it can independently perform data transfer operations without CPU participation. At the same time, the CPU can perform other arithmetic operations, which can improve the parallelism of the system, and increase the data throughput rate. All these factors can enhance the system performance [3]. The DMA controller is essentially a special interface circuit [4], the function is to generate read and write addresses. Then it reads the data from the source device to the internal FIFO, the data of which will be transferred to the destination device. It exerts its role as a bridge for data transfer between internal memory and peripherals [5].

This paper introduces a design technology of a DMA controller based on MCU. The controller supports 8 transfer channels, 4 transfer methods between memory and peripheral, data width of the source and destination configured as byte, halfword and word 3 types. It also supports burst types of transfer configured as single, burst4, burst8 and burst16. The controller can achieve channel switching after accomplishing a DMA transfer (once source transfer and destination transfer). Furthermore, this paper proposes an approach for crossing the 1K address boundary during multi-beat burst transfer and a processing method for the remaining data that does not meet the configured burst transfer type.

2. Design architecture of DMA controller

The overall architecture of the DMA controller is demonstrated in Figure 1. It is mainly composed of

Content from this work may be used under the terms of the Creative Commons Attribution 3.0 licence. Any further distribution of this work must maintain attribution to the author(s) and the title of the work, journal citation and DOI.
Published under licence by IOP Publishing Ltd

AHB-Lite Slave module, channel request module, arbitration module, transfer control module, interrupt generation module, handshake module, AHB-Lite master module and other sub-modules.

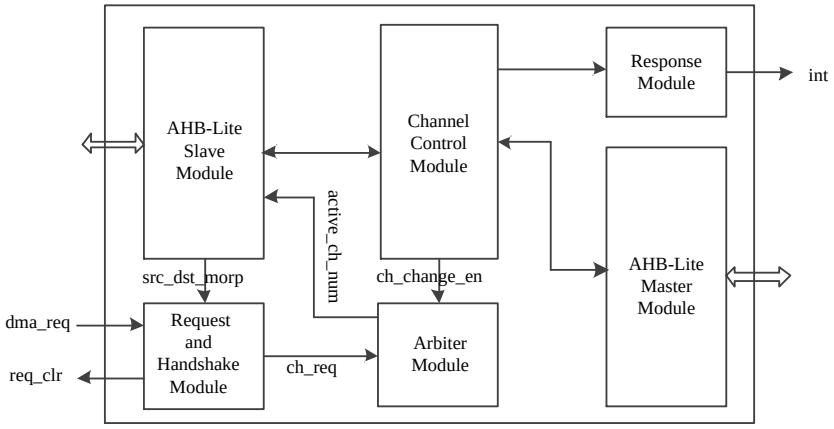

Fig.1 overall architecture of DMA controller

AHB-Lite slave module is applied in the function configuration from CPU. It transmits the configuration information of the current transfer channel to the transfer control and the AHB-Lite master module. In the meantime, relevant registers will be updated in accordance with the feedback of these two modules. Channel request module generate or process the transfer request signal according to the source and destination devices and send it to the arbitration module, which arbitrates the highest priority request based on the transfer request signal of each channel to obtain the transfer control right. And the transfer control module controls the data transfer process from the source device to the destination device. The function of the handshake module is to generate clear signal to clear request signal and sends it to peripheral unit when completing a burst transfer or all data of the transferring channel is completed. The AHB-Lite master module transmits the address, control information to the bus according to the requirements of channel transfer and the AHB-Lite protocol specification and performs source data reception and destination data transmission.

3. The Concrete Realization of DMA Controller

3.1 Design of Request Module and Handshake Module

In the design, the source and destination transfer both support two type devices: memory and peripherals. When the transfer device is a peripheral, it will input a transfer request signal to the controller to request data transfer, and most of the peripherals are integrated in the lower frequency APB bus, so the controller has the demand for the process of crossing clock domain when receiving a peripheral request to minimize effect of metastability. In this design, a two-stage synchronization method is deployed to synchronize the request signal to the controller clock domain. The memory cannot generate a transfer request signal, if the transfer device is a memory, the request signal should be generated in this module, which means when the source or destination device is configured as the memory the software, the source or destination request defaults to be valid. In the meantime, in order to reduce the response time of the DMA to the request, the signal and the DMA don’t need synchronization. The source request and the destination request are combined to get the request signal to the arbitration module in order to reduce the long-term occupation of the DMA controller on the bus, and the request module structure is shown in the figure 2.

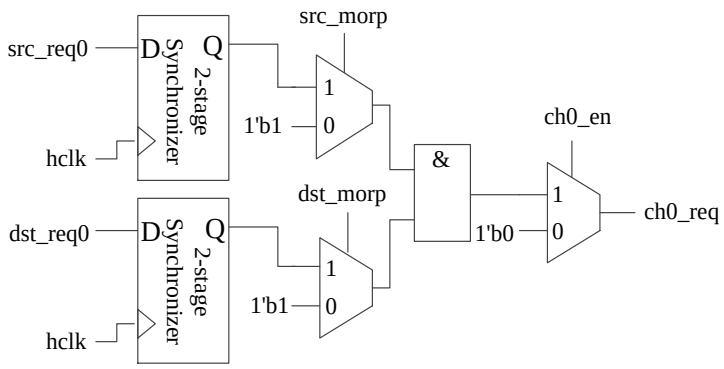

Fig.2 Request module structure diagram

The function of handshake module is to generate a request clear signal and make output to peripherals, clearing peripheral request signal when the transfer is completed. After the corresponding peripheral request signal is cleared, the request signal is fed back to the handshake module, which clears the request clear signal. And the structure of the handshake module is shown in Figure 3.

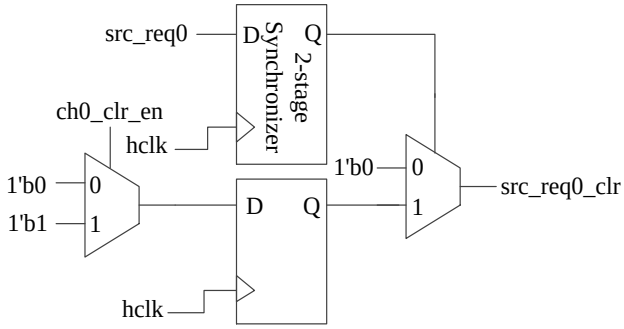

Fig.3 Handshake module structure diagram

3.2 Design of Arbitration Module and Channel Transmission Control Module

In the arbiter module, the arbitration algorithm adopts a fixed priority algorithm. The priority of each channel decreases as the channel number descends, which means channel 7 has the highest priority, and channel 0 has the lowest priority. When multiple channel request signals are valid simultaneously, this module will select the channel whose request signal is valid and which has the highest channel number to obtain the transfer control right. The higher-priority channel can preempt the transfer control after a DMA transfer is completed. For example, when channel 3 is transmitting data, the request signal of channel 5 is valid. Channel 5 will obtain the transfer control right after the channel 3 completed of the current burst transfer in the system. After the data transfer of channel 5 is completed, it then returns the transfer control unit to the channel 3 to continue the transfer that has not been finished before channel 3 is preempted.

The transfer control module is the core module in this design. Containing 3 sub-modules, and they are the transfer state machine module, the control information recalculation and the FIFO module respectively.

The main function of the transfer state machine module is to control and coordinate other modules to complete the entire transfer process. In this module, a whole DMA transfer includes three stages: the recalculation, the source transfer and the destination transfer stage, and then the source and destination transfer stages are subdivided into several states, which are shown in the figure 4. In the LOOP_RELOAD state, the control signal is recalculated on the basis of the data width, burst type and transfer address of the current transfer channel's source and destination transfer. All these generate the

control signal conforming to the AHB-Lite protocol and configuration information and some of them are discussed specifically in next parts.

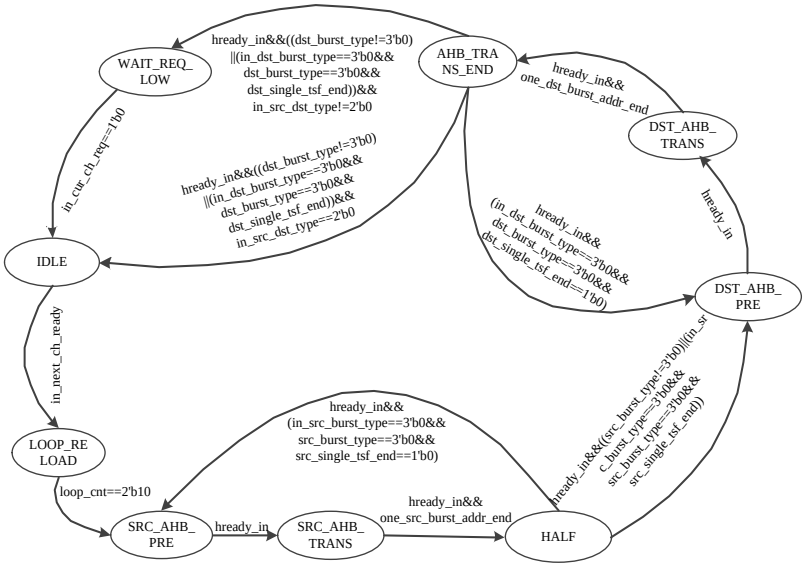

Fig.4 State transition diagram of transfer state machine

The control information calculation module is the key design of this controller. Its main feature is to recalculate based on the information transmitted into this module from the AHB-Lite slave module. And it attempts to calculate burst types of the source and destination transfer. The type of burst will get changes in two situations: one is that next multi-beat burst transfer will cross the 1K address boundary. The other is that the number of remaining un-transferred data is less than the length of the configured burst type. While burst type is single-beat or transfer address is fixed, it is unnecessary to care the condition of crossing the 1K address boundary. When multi-beat burst transfer is performed and the transfer address is incremented, it is crucial to calculate the address after a burst transfer is completed according to the current transfer data width and the type of burst. If the lower 10bits of the address is greater than the lower 10bits of the address before transfer, it means that the burst transfer will not cross the 1K address boundary, and there is no need to change the burst type. Otherwise, it implies that the 1K address boundary will be crossed. And in this case, multi-beat burst transfer is required to renew to several single-beat burst transfer to cross the 1K address boundary, besides, the number of single-beat burst transfer is up to the configured burst type. If the number of remaining un-transmitted data is less than the configured one burst type and does not cross the 1K address boundary, then the burst type is refreshed to a burst transfer with indefinite length, otherwise the burst transfer type is updated to several single-beat burst to cross the 1K address boundary. When the 1K address boundary is crossed and number of remaining un-transmitted data is greater than the beats of burst transfer type, the data is transferred according to the configured burst type.

The FIFO module is applied to store the source data in transfer process. In the source transfer stage, data is transferred from the source device to the FIFO, and then the data in the FIFO is transferred to the destination device in the destination transfer stage. In this design, the source data width and the destination data width and transfer size are configurable, so the data will be packed and unpacked in the FIFO module. The data in the FIFO is stored in the little-endian mode. That means if the source device data is reserved in big-endian mode, which must be converted to little-endian mode when stored in FIFO.

3.3 AHB-Lite Master Module Design

This module mainly generates address and control information that comply with the AHB-Lite protocol based on the current configuration of the transfer channel and the update situation across the

1K boundary. At the end of the source transfer or destination transfer, the corresponding address is output to the AHB-Lite slave module, renewing the relevant register information of the current transfer channel.

4. Verification of DMA Controller

This design adopts a SoC system based on ARM Cortex-M3 to verify the function of DMA controller. The system architecture is shown in Figure 5. The simulation verification was carried out with Modelsim, which mainly verifies the functions of DMA controller's reading or writing transfer, transferring based on crossing 1K address boundary, software and hardware triggering transfer, and transfer channel switching.

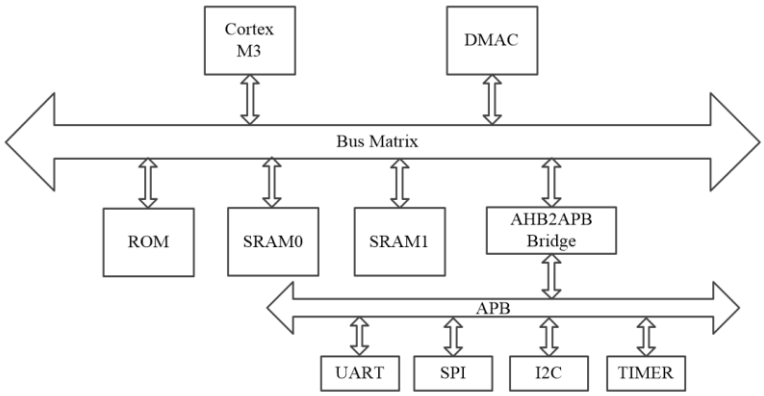

Fig.5 environment diagram of DMA controller verification

Figure 6 depicts the higher-priority channel is preempting the channel control module to transfer data. It can be confirmed when channel 0 is in the transmission process, channel 1 is sending a transfer request. After the completion of the current DMA transfer of channel 0, channel 1 is starting to transfer data. After the data transfer of channel 1 is finished, channel 0 obtains the transfer control right to transfer the remaining data again.

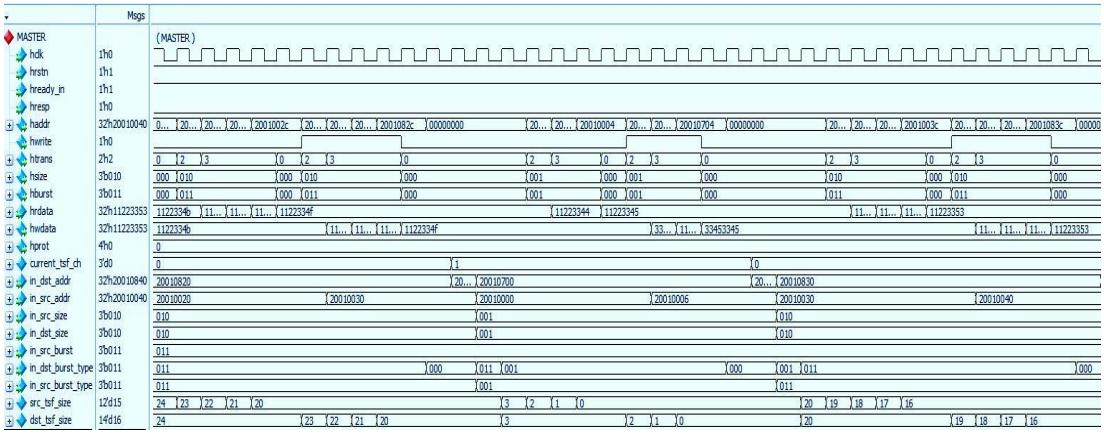

Fig.6 simulation diagram of channel switching

Figure 7 is the functional simulation when crossing the 1K address boundary during the destination transfer. The results indicate that when a transfer crosses 1K address boundary, the multi-beat burst transfer will be converted into several single-beat burst transfer to cross the 1K address boundary.

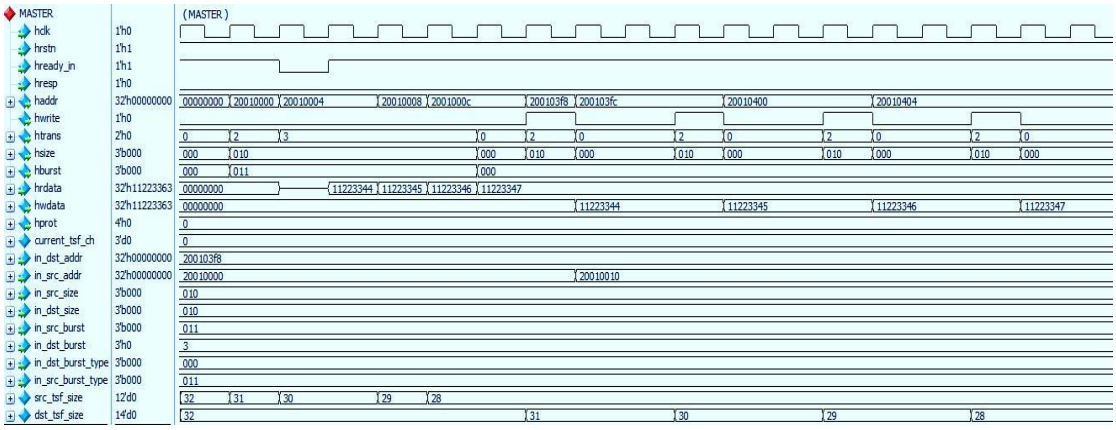

Fig.7 Simulation diagram of crossing 1K address boundary

5. Conclusion

This paper proposes a design method of an 8-channel DMA controller based on AHB-Lite protocol in response to the MCU's requirements for data transfer performance improvement, which attributes the completion of RTL design and function verification of Controller. The results signifies that the controller makes improvement of the efficiency of data transmission in MCU system, reducing the time in the process of transporting data in the CPU and enhancing system performance. Presently, the design has been successfully applied to the motor drive chip based on Cortex-M3.

References

[1] Wang, Y, Wang, T, Zhou, P. (2014) Design and Implementation of a Flexible DMA Controller in Video Codec System. In: Proceedings of the 19th International Conference on Digital Signal Processing, Hong Kong. pp. 78-82.

[2] Ahmed, M.A., Aljumah, A., Ahmad, M.G. (2019) Design and Implementation of a Direct Memory Access Controller for Embedded Applications. J. International Journal of Technology. 10(2): 309-319.

[3] Shirur, Y.J.M., Sharma, K.M., A, A.(2018) Design and implementation of Efficient Direct Memory Access (DMA) Controller in Multiprocessor SoC. In: International Conference on Networking, Embedded and Wireless Systems (ICNEWS). Bangalore. pp. 1-6.

[4] Gao, Y. X. The Design and Implementation of DMA Controller based on AHB Bus[D]. Shenyang: Liaoning University,2020.

[5] Zhang, D.Y. The Analysis and Verification of Direct Memory Access Controller based on AHB[D]. Xi'an: Xidian University，2015.